算法概述

1A. 算法

- 算法要素：输入，输出，正确，明确，可行，有穷

1C.时间复杂度

- 大O记号：
  - $T(n) \leq cf(n) \Rightarrow T(n) = O(f(n))$
- 大Ω记号：
  - $T(n) \geq cf(n) \Rightarrow T(n) = \Omega(f(n))$
- 大Θ记号：
  - $c_1 f(n) \leq T(n) \leq c_2 f(n) \Rightarrow T(n) = \Theta(f(n))$
- $log^* n$：一直对n取对数，直到小于2的操作次数，成为迭代对数
- 复杂度排序：$O(1) < O(\alpha(n)) < O(log^* n) < O(loglogn) < O(logn) < O(\sqrt{n}) < O(n)$

1D.复杂度分析

- 大师定理：$T(n) = aT(n/b) + \Omega(g(n))$
  - 比较$n^{log_b a}$和$g(n)$,谁大是谁，几乎一致时为若$g(n) = n^{log_b a} log^k n$,则为$n^{log_b a} log^{k+1} n$
- Akra-Bazzi定理：$T(n) = \sum_{i=1}^{k} a_i T(b_i n + h_i(n)) + \Omega(g(n))$
  - 取p使得$\sum_i^k a_i b_i^p = 1$,$T(n) = \Theta(n^p(1 + \int_1^n g(u)/u^{p+1} du))$

1E.迭代和递归

- 最大总和区间：减治策略，从末尾开始找，$O(n)$；分治为$O(nlogn)$

1E.动态规划

- 记忆化：时间复杂度$O(mn)$

向量

2C.有序向量

- 去重只需$O(n)$
- 二分查找$O(logn)$：
  - 平均查找长度，方案C的长度为$log_2 n$
  - B方案search(e)接口返回不大于e的最靠后的元素
- 插值查找
  - 时间复杂度为$O(loglogn)$

2D.排序

- 起泡排序：都需要$\Theta(n^2)$
- 提前终止版和跳跃版

2E.归并排序

- 最坏情况最优的算大，根据分治实现

2F.位图

- 处理由若干个非负整数构成的集合

列表

3A.接口与实现

- pred，succ为前驱和后继
- 哨兵节点：头节点和尾结点

3B.无序列表

- 查找：自后向前
- 去重：输入敏感

3C.有序列表

- 查找：只能顺序查找，效率为$O(n)$

3D.选择排序

- 稳定为$O(n^2)$，但是常数略好于冒泡

3E.插入排序

- 输入敏感
- 在线算法/联机算法

3F.归并排序

- 统计逆序对：时间复杂程和归并相同
- 冒泡，选择，插入只能处理相邻逆序对，所以时间复杂度不低于逆序对格数

栈和队列

4A.栈

- pop弹出栈顶元素，top引用，push插入

4B.调用栈

- 空间复杂度也即调用栈规模最大的时候

4F.栈混洗

- 卡特兰数：$SP(n) = (2n)!/(n+1)!/n!$
- 禁形：ijk$\rightarrow$kij
- n个元素的栈混洗和n对括号构成的匹配表达式是一一对应的

4G.队列

- enqueue插入，dequeue删除

4I.最大矩形

- 栈的大小期望规模为$O(logn)$
- 单次扫描在线

二叉树

5B.二叉树（binary tree）

- 真二叉树：不含单分支节点的二叉树
- 满二叉树：全部填满
- 完全二叉树：最底一层可空缺，其余不可

5C.遍历

- 后序遍历：任何一个合法表达式都对应一棵表达式树，内部节点为运算符，叶节点为运算数
- succ()接口：有右孩子右孩子的最左，没右孩子左祖先到头的右祖先
- 层次遍历：先上后下，先左后右
- 完全二叉树内部节点数量为$\lfloor n/2 \rfloor$

5D.Huffman编码